

Temporary slide

Topics to cover in order, the specific details are not exhaustive)

1. Introduction (what it is, and why it came to be such as low power usage market etc etc)
2. ISA (explain its x86 compatibility through CMS and what the processor truly is behind the CMS, VLIW)
3. Pipeline (show its pipeline stages, and how instructions pass through, Branch prediction etc)
4. Register file and precise exceptions (working/shadow registers, committing, and what happens when CMS fails)
5. Memory (Memory model, Cache hierarchy, how the cache works, memory consistency, TLB etc)
6. Performance / Limitations (Why it was so slow, benchmarks compared to other CPUs at the time)

REMOVE BEFORE PRESENTATION

Transmeta Crusoe

Arunachalam, Valli
Tarra, Sai
Chen, Hai-Bi

Agenda

- | | |
|---|--------|
| 1. Introduction(feature and structure) | P.1-5 |
| 2. ISA (explain its x86 compatibility through CMS and what the processor truly is behind the CMS, VLIW) | P.7-12 |
| 3. Pipeline (show its pipeline stages, and how instructions pass through, Branch prediction etc) | P.13-1 |
| 4. Register file and precise exceptions (working/shadow registers, committing, and what happens when CMS fails) | P.1-1 |
| 5. Memory (Memory model, Cache hierarchy, how the cache works, memory consistency, TLB etc) | P.1-1 |
| 6. Performance / Limitations (Why it was so slow, benchmarks compared to other CPUs at the time) | P.1-1 |
| 7. Question | P.1-1 |

Introduction

Feature

An x86-compatible family of solutions

Strong performance, low power consumption

How is it achieved?

Composed of a hardware engine and a software layer

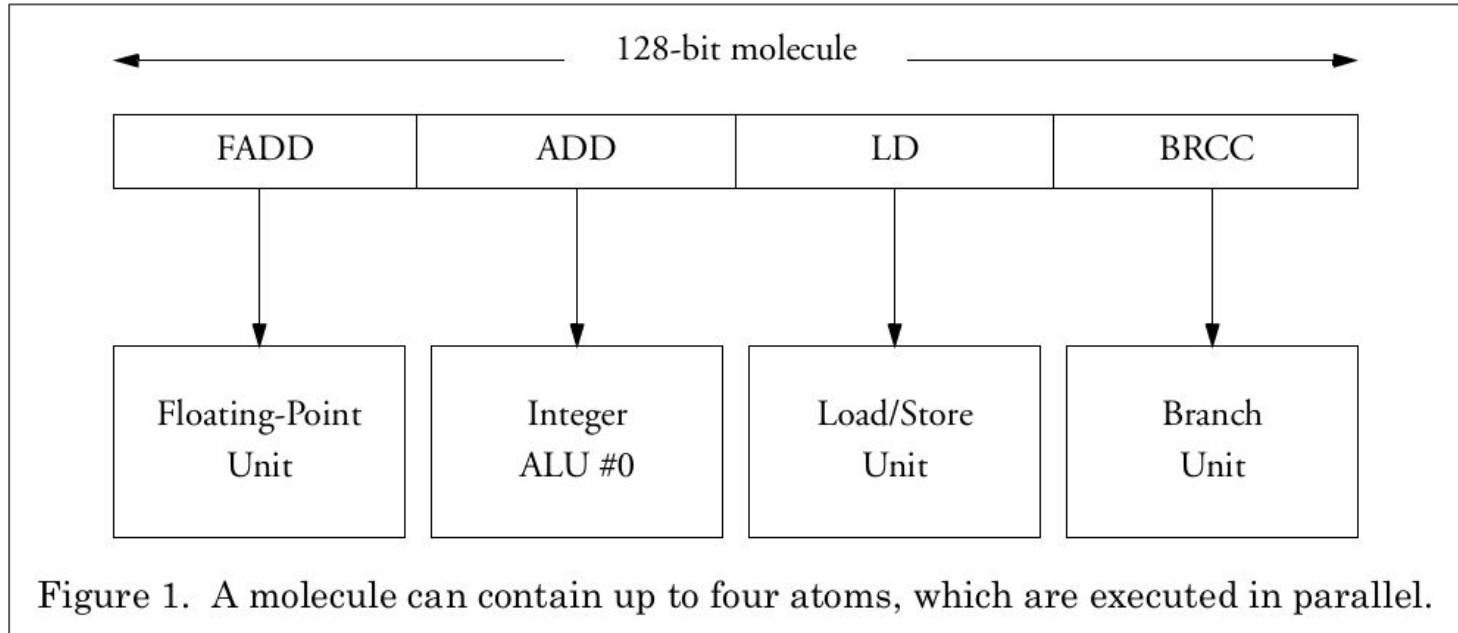
Introduction

Structure

- VLIW
- The Code Morphing Software
- Shadowed registers
- Longrun Power Management

Introduction

VLIW



Introduction

The Code Morphing Software

Shadowed registers

Introduction

Longrun Power Management

ISA - Code Morphing Software

The Interesting part of the ISA is the Code Morphing Software (CMS).

The Code Morphing Software is a Dynamic Translator that converts X86 instructions into VLIW instructions.

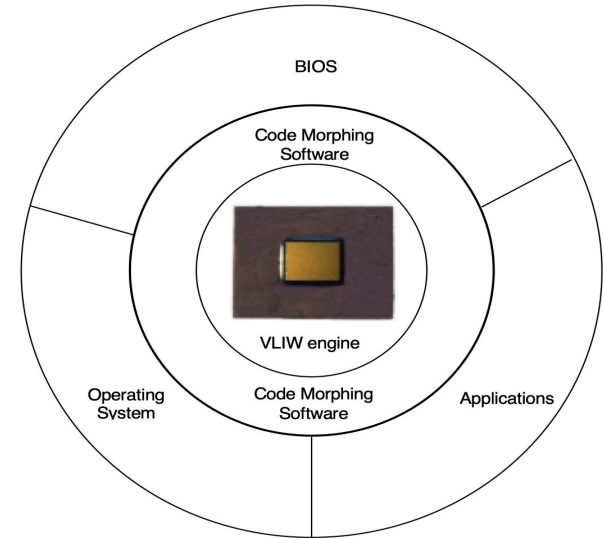


Figure 5. The Code Morphing software mediates between x86 software and the Crusoe processor.

Figure from [1]

ISA - VLIW Instructions

VLIW - Very Long Instruction Word Architecture

Focuses on achieving high levels of instruction level parallelism.

Atoms - each individual instruction is essentially an atom

Molecule - group of atoms

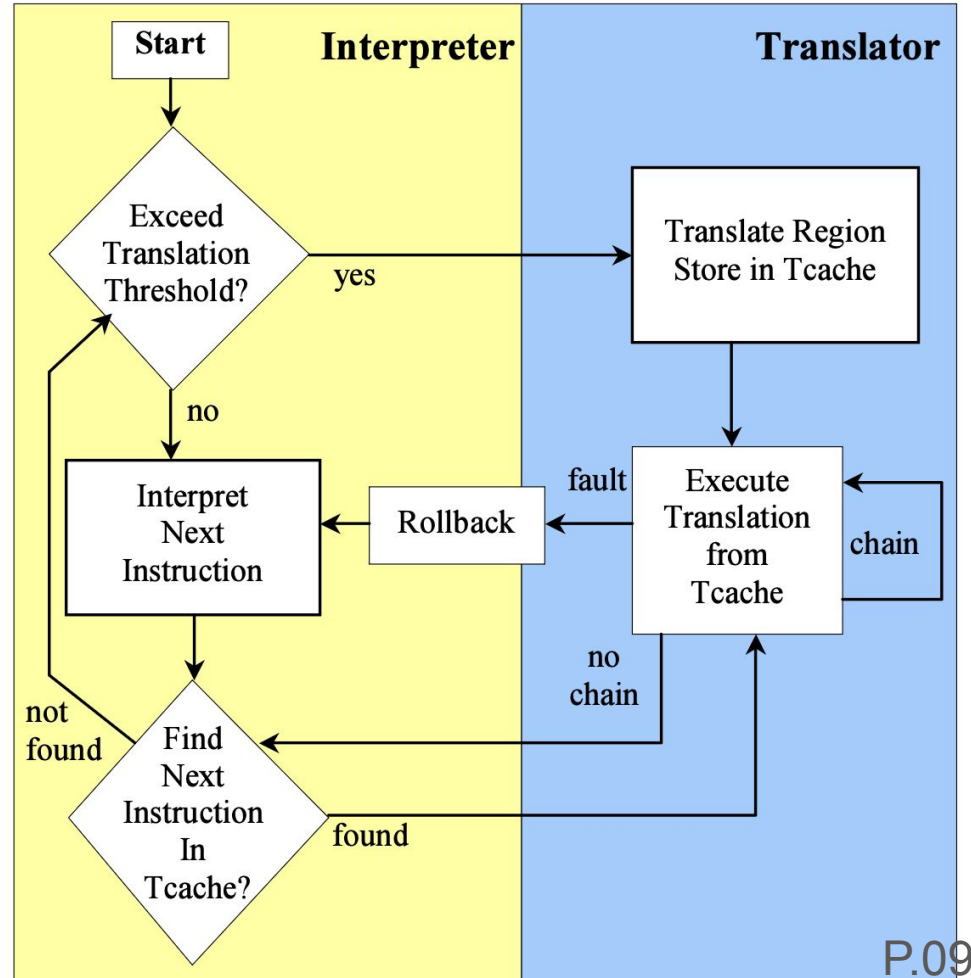
A VLIW compiler scans through the code to analyze which atoms can be grouped together

These instructions are paired together to achieve a long instruction (a molecule) that the processor can then execute.

CMS Workflow

Interpreter - Processes the program one instruction at a time. Take a x86 instruction and emulates it in software.

Translator - Translates chunks of frequently processed code and stores them in the Translation Cache (Tcache).



Example

Original X86 instructions

- A. `addl %eax, (%esp)` `//load data from stack and add to %eax`
- B. `addl %ebx, (%esp)`
- C. `movl %esi, (%ebp)` `//load %esi from memory`
- D. `subl %ecx, 5` `//subtract 5 from %ecx register`

Translated instructions

```
ld %r30, [%esp]            //load from stack into temporary registers
add.c %eax, %eax, %r30
ld %r31, [%esp]
add.c %ebx, %ebx, %r31
ld %esi, [%ebp]
sub.c %ecx, %ecx, 5
```

Example continued

Translated instructions

```
ld %r30,[%esp]      //load from stack into temporary registers
add.c %eax,%eax,%r30
ld %r31,[%esp]
add.c %ebx,%ebx,%r31
ld %esi,[%ebp]
sub.c %ecx,%ecx,5
```

Optimized instructions

```
ld %r30,[%esp]
add %eax,%eax,%r30
add %ebx,%ebx,%r30
ld %esi,[%ebp]
sub.c %ecx,%ecx,5
```

Example continued

Optimized instructions

```
ld %r30,[%esp]
add %eax,%eax,%r30
add %ebx,%ebx,%r30
ld %esi,[%ebp]
sub.c %ecx,%ecx,5
```

Final VLIW instructions

1. ld %r30,[%esp]; sub.c %ecx,%ecx,5
2. ld %esi,[%ebp]; add %eax,%eax,%r30; add %ebx,%ebx,%r30

Pipeline - Complexity

Traditional x86 CPU

- Complex decode hardware
- Complex scheduling hardware
- Simpler compiler
- Complex pipeline

VLIW like Itanium

- Simple decode hardware
- Complex compiler
- Simple pipeline

Crusoe

- Simple decode hardware
- CMS does all the complex scheduling work
- Simpler compiler
- Simple pipeline

Takeaway:

- CMS software takes all the complexity
- Keeping the compiler AND pipeline simple

Pipeline stages

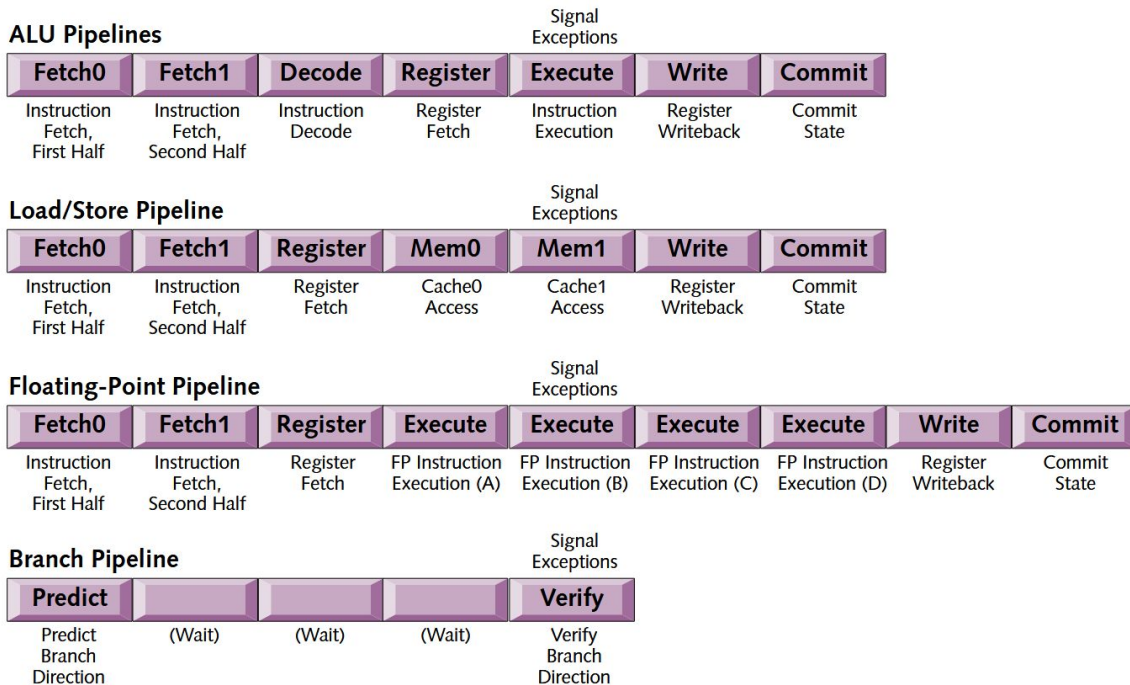
The pipeline contains 5 total execution units:

- 2x ALU
- 1x Load/Store
- 1x FP
- 1x Branch

Up to 4 instructions can be fed per cycle:

- 1 Load/Store
- 1 Branch
- 1 ALU
- 1 FP or ALU

Relies on the CMS to schedule molecules such that each one has a Load/Store, Branch etc etc.

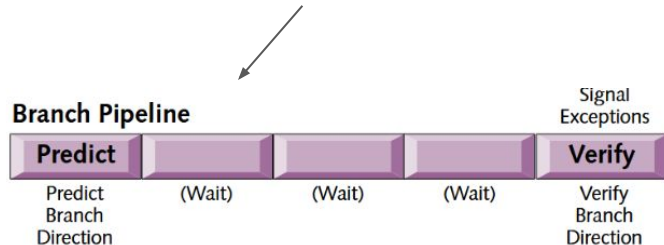


Branch Prediction

Once again CMS took on more complexity.

- Hardware doesn't contain branch prediction
- CMS analysed the x86 code logic as it was being interpreted (Slow)
- Dynamically profiled the code. (Branch x is taken 90% of the time, then branch y is taken 99% of the time etc etc)
- CMS translated the common x86 paths into blocks of native atoms and molecules
- Stored these blocks of translated code into the Translation Cache.
- Next time it runs across the same code, it can execute the translated block (Fast)

This keeps the branch unit very simple and acts as a way to verify the assumptions made by software.



Register file

64, 32-bit general-purpose registers

48, shadowed 32-bit general-purpose registers

32, 80-bit floating-point registers

16, shadowed 80-bit floating-point registers

Why so many?

1. Used to store x86 state
2. CMS software needs to run too
3. CMS renames x86 registers to reduce dependencies for better VLIW scheduling
4. Storing a checkpoint (Commit and rollback to deal with exceptions)

```
mov eax, 10    =>  mov r1, 10
add eax, 5      =>  mov r2, 5
add eax, 2      =>  mov r3, 2
add eax, 3      =>  mov r4, 3
```

Now CMS can pack them all into a molecule

Precise Exceptions

Now we have to deal with normal x86 exceptions, (Ex. Page faults) AND any generated by CMS translation.

- CMS made many assumptions during translations such as reordering instructions to fit more into molecules. It may reorder a store and a load since it thinks the the memory addresses are different.

Example:

X86:

store [EAX]
load [EBX]



CMS assumes
EAX != EBX



Native Code (not truly accurate):

load[EBX]
...
...
store [EAX]



Alias detection
hardware raises a
fault. What now?

Precise Exceptions

- Shadow registers keep a committed copy of a known good program state
- Gate store buffer keeps all stores and loads temporarily

When a translation unit successfully executes:

1. all working registers are copied into shadow registers
2. All stores and loads in gate store buffer are pushed into the normal memory hierarchy

When a translation faults:

1. Revert to the known good state from the shadow registers and flush gate store buffer.
2. Run the x86 code by interpreting without any CMS tricks, if it faults again, it's a typical x86 exception (div by 0, page fault)
3. If it doesn't fault, the CMS's translation is the issue and execution can continue

Memory

Cache

The translation cache, along with the Code Morphing code, resides in a separate memory space that is inaccessible to x86 code.

How does it work?

Takes advantage of locality of reference

Memory

Alias Hardware

To break limits imposed by memory dependencies

How does it work?

Store-under-alias-maks

Memory

Coping with self-modifying code

Addressing potential overwrites of translated x86 instructions

How it works?

Write-protecting x86 memory pages containing the code

Limitations

The crusoe processor was slow especially with floating point instructions. About 6 times slower execution time for floating point instructions than C3 Nehemiah on the floating point multimedia benchmark.

The processor might not always be able to handle 4 atoms at a time leading to inefficiencies.

Insufficient L2 cache which results in cache thrashing.

Huge translation penalty.

Limited clock speed.

Discussion Questions

How would a program with multiple branches affect the processors performance?

Would a larger L2 cache support performance?

Could the dynamic translation approach outperform traditional execution?

References

[1] https://mark.people.clemson.edu/330/paper_aklaiber_19jan00.pdf

[2]